

Fall 2025 CPTS530 Homework 5
MATH 448-548
Numerical Analysis

Aryan Ritwajeet Jha

November 2025

Contents

Problem 1	2
Problem 2	4
Problem 3	5
Problem 4	6
Problem 5	8
Problem 6	11
Problem 7	14
Problem 8	16
Problem 9	18
Appendix	19
References	21

Problem 1

Problem Statement (Textbook 4.4.1):

Show that the norms $\|\mathbf{x}\|_\infty$, $\|\mathbf{x}\|_2$, and $\|\mathbf{x}\|_1$ satisfy the postulates (1), (2), and (3) for norms.

Norm Postulates [1, 2]:

On a vector space V , a **norm** is a function $\|\cdot\|$ from V to the set of nonnegative reals that obeys these three postulates:

$$\|x\| > 0 \text{ if } x \neq 0, x \in V \quad (1)$$

$$\|\lambda x\| = |\lambda| \|x\| \text{ if } \lambda \in \mathbb{R}, x \in V \quad (2)$$

$$\|x + y\| \leq \|x\| + \|y\| \text{ if } x, y \in V \quad (\text{triangle inequality}) \quad (3)$$

Solution:

We verify postulates (1), (2), and (3) for $\|x\|_\infty = \max_i |x_i|$, $\|x\|_2 = \sqrt{\sum_i x_i^2}$, and $\|x\|_1 = \sum_i |x_i|$.

(a) ∞ -norm: $\|x\|_\infty = \max_{1 \leq i \leq n} |x_i|$

(1) If $x \neq 0$, then $\exists i$ with $x_i \neq 0$, so $|x_i| > 0$. Thus $\|x\|_\infty = \max_i |x_i| > 0$.

(2) $\|\lambda x\|_\infty = \max_i |\lambda x_i| = \max_i |\lambda| |x_i| = |\lambda| \max_i |x_i| = |\lambda| \|x\|_\infty$.

(3) $\|x + y\|_\infty = \max_i |x_i + y_i| \leq \max_i (|x_i| + |y_i|) \leq \max_i |x_i| + \max_i |y_i| = \|x\|_\infty + \|y\|_\infty$.

(b) 2-norm (Euclidean): $\|x\|_2 = \sqrt{\sum_{i=1}^n x_i^2}$

(1) If $x \neq 0$, then $\exists i$ with $x_i \neq 0$, so $\sum_i x_i^2 > 0$. Thus $\|x\|_2 = \sqrt{\sum_i x_i^2} > 0$.

(2) $\|\lambda x\|_2 = \sqrt{\sum_i (\lambda x_i)^2} = \sqrt{\lambda^2 \sum_i x_i^2} = |\lambda| \sqrt{\sum_i x_i^2} = |\lambda| \|x\|_2$.

(3) Expanding $\|x + y\|_2^2$:

$$\begin{aligned} \|x + y\|_2^2 &= \sum_i (x_i + y_i)^2 = \sum_i x_i^2 + 2 \sum_i x_i y_i + \sum_i y_i^2 \\ &= \|x\|_2^2 + 2 \sum_i x_i y_i + \|y\|_2^2 \end{aligned}$$

Since $\sum_i x_i y_i \leq \|x\|_2 \|y\|_2$ (dot product bounded by product of norms):

$$\|x + y\|_2^2 \leq \|x\|_2^2 + 2\|x\|_2 \|y\|_2 + \|y\|_2^2 = (\|x\|_2 + \|y\|_2)^2$$

Taking square roots: $\|x + y\|_2 \leq \|x\|_2 + \|y\|_2$.

(c) 1-norm: $\|x\|_1 = \sum_{i=1}^n |x_i|$

(1) If $x \neq 0$, then $\exists i$ with $x_i \neq 0$, so $|x_i| > 0$. Thus $\|x\|_1 = \sum_i |x_i| > 0$.

(2) $\|\lambda x\|_1 = \sum_i |\lambda x_i| = \sum_i |\lambda| |x_i| = |\lambda| \sum_i |x_i| = |\lambda| \|x\|_1$.

(3) For each term, the triangle inequality for absolute values gives $|x_i + y_i| \leq |x_i| + |y_i|$.

Summing over all i :

$$\|x + y\|_1 = \sum_i |x_i + y_i| \leq \sum_i (|x_i| + |y_i|) = \sum_i |x_i| + \sum_i |y_i| = \|x\|_1 + \|y\|_1$$

All three norms satisfy postulates (1), (2), and (3).

Problem 2

Problem Statement (Textbook 4.4.2):

Show that $\|\mathbf{x}\|_\infty \leq \|\mathbf{x}\|_2 \leq \|\mathbf{x}\|_1$ for all $\mathbf{x} \in \mathbb{R}^n$, and that equalities can occur, even for nonzero vectors.

Solution:

Part 1: Prove $\|x\|_\infty \leq \|x\|_2$

Let $\|x\|_\infty = \max_i |x_i| = |x_k|$ for some index k . Then:

$$\|x\|_2^2 = \sum_{i=1}^n x_i^2 \geq x_k^2 = \|x\|_\infty^2$$

Taking square roots: $\|x\|_\infty \leq \|x\|_2$.

Equality case: Equality holds when x has only one nonzero component. For example, $x = (5, 0, 0)^T$ gives $\|x\|_\infty = 5$ and $\|x\|_2 = 5$.

Part 2: Prove $\|x\|_2 \leq \|x\|_1$

First, observe that each component satisfies $|x_i| \leq \|x\|_1$ (since $\|x\|_1 = \sum_{j=1}^n |x_j|$ includes all terms).

For each i , we can write:

$$|x_i|^2 = |x_i| \cdot |x_i| \leq |x_i| \cdot \|x\|_1$$

where the inequality holds because $|x_i| \leq \|x\|_1$.

Now sum this inequality over all indices:

$$\begin{aligned} \|x\|_2^2 &= \sum_{i=1}^n |x_i|^2 \leq \sum_{i=1}^n |x_i| \cdot \|x\|_1 \\ &= \|x\|_1 \sum_{i=1}^n |x_i| \quad (\text{factor out } \|x\|_1) \\ &= \|x\|_1 \cdot \|x\|_1 = \|x\|_1^2 \end{aligned}$$

Taking square roots of both sides: $\|x\|_2 \leq \|x\|_1$.

Equality case: Equality holds when x has only one nonzero component. For example, $x = (0, 3, 0)^T$ gives $\|x\|_2 = 3$ and $\|x\|_1 = 3$.

Conclusion: The inequality $\|x\|_\infty \leq \|x\|_2 \leq \|x\|_1$ holds for all $x \in \mathbb{R}^n$, with equality possible for nonzero vectors.

Problem 3

Problem Statement (Textbook 4.4.3):

(Continuation) Show that $\|\mathbf{x}\|_1 \leq n\|\mathbf{x}\|_\infty$ and $\|\mathbf{x}\|_2 \leq \sqrt{n}\|\mathbf{x}\|_\infty$ for $\mathbf{x} \in \mathbb{R}^n$.

Solution:

Part 1: Prove $\|x\|_1 \leq n\|x\|_\infty$

Since $\|x\|_\infty = \max_i |x_i|$, we have $|x_i| \leq \|x\|_\infty$ for each $i = 1, 2, \dots, n$.

Summing over all components:

$$\|x\|_1 = \sum_{i=1}^n |x_i| \leq \sum_{i=1}^n \|x\|_\infty = n\|x\|_\infty$$

Therefore: $\boxed{\|x\|_1 \leq n\|x\|_\infty}$.

Equality case: Equality holds when all components have the same absolute value. For example, $x = (1, 1, 1)^T$ gives $\|x\|_1 = 3 = 3 \cdot 1 = 3\|x\|_\infty$.

Part 2: Prove $\|x\|_2 \leq \sqrt{n}\|x\|_\infty$

Since $|x_i| \leq \|x\|_\infty$ for each i , squaring gives $|x_i|^2 \leq \|x\|_\infty^2$.

Summing over all components:

$$\|x\|_2^2 = \sum_{i=1}^n |x_i|^2 \leq \sum_{i=1}^n \|x\|_\infty^2 = n\|x\|_\infty^2$$

Taking square roots: $\boxed{\|x\|_2 \leq \sqrt{n}\|x\|_\infty}$.

Equality case: Equality holds when all components have the same absolute value. For example, $x = (2, 2, 2)^T$ gives $\|x\|_2 = \sqrt{12} = 2\sqrt{3} = \sqrt{3} \cdot 2 = \sqrt{3}\|x\|_\infty$.

Problem 4

Problem Statement (Textbook 4.4.8):

Define

$$\|A\| = \sum_{i=1}^n \sum_{j=1}^n |a_{ij}|$$

Show that this is a matrix norm (that is, a norm on the linear space of all $n \times n$ matrices). Show that it is not subordinate to any vector norm. Does it conform to Equation (9) and Inequality (10)?

$$\|I\| = 1 \quad (\text{Equation 9})$$

$$\|AB\| \leq \|A\|\|B\| \quad (\text{Inequality 10})$$

Solution:

Part 1: Show $\|A\|$ is a matrix norm

A matrix norm must satisfy three postulates:

- (1) **Positivity:** If $A \neq 0$, then $\exists$ some $a_{ij} \neq 0$, so $|a_{ij}| > 0$. Thus $\|A\| = \sum_{i,j} |a_{ij}| > 0$.
- (2) **Homogeneity:** $\|\lambda A\| = \sum_{i,j} |\lambda a_{ij}| = \sum_{i,j} |\lambda| |a_{ij}| = |\lambda| \sum_{i,j} |a_{ij}| = |\lambda| \|A\|$.
- (3) **Triangle inequality:**

$$\begin{aligned} \|A + B\| &= \sum_{i,j} |a_{ij} + b_{ij}| \leq \sum_{i,j} (|a_{ij}| + |b_{ij}|) \\ &= \sum_{i,j} |a_{ij}| + \sum_{i,j} |b_{ij}| = \|A\| + \|B\| \end{aligned}$$

Therefore, $\|A\|$ is a matrix norm.

Part 2: Show it is NOT subordinate to any vector norm

A subordinate (induced) matrix norm satisfies $\|I\| = 1$. However, for the identity matrix I where $(I)_{ij} = \delta_{ij}$ (Kronecker delta: $\delta_{ij} = 1$ if $i = j$, else 0):

$$\|I\| = \sum_{i=1}^n \sum_{j=1}^n |\delta_{ij}| = \sum_{i=1}^n 1 = n$$

Since $\|I\| = n \neq 1$ for $n > 1$, this norm is NOT subordinate to any vector norm.

Part 3: Does it conform to Equation (9)?

Equation (9) requires $\|I\| = 1$. As shown above, $\|I\| = n$.

No, it does NOT conform to Equation (9).

Part 4: Does it conform to Inequality (10)?

We need to check if $\|AB\| \leq \|A\|\|B\|$.

Step 1: We recall that for matrix multiplication, the (i, k) entry of AB is:

$$(AB)_{ik} = \sum_{j=1}^n a_{ij}b_{jk}$$

Step 2: Using the triangle inequality for absolute values, we have:

$$|(AB)_{ik}| = \left| \sum_{j=1}^n a_{ij}b_{jk} \right| \leq \sum_{j=1}^n |a_{ij}b_{jk}| = \sum_{j=1}^n |a_{ij}||b_{jk}|$$

Step 3: We now compute $\|AB\|$ by summing over all entries:

$$\begin{aligned} \|AB\| &= \sum_{i=1}^n \sum_{k=1}^n |(AB)_{ik}| \\ &\leq \sum_{i=1}^n \sum_{k=1}^n \sum_{j=1}^n |a_{ij}||b_{jk}| \quad (\text{from Step 2}) \end{aligned}$$

Step 4: We rearrange the triple sum. Notice that $|a_{ij}|$ doesn't depend on k , and $|b_{jk}|$ doesn't depend on i :

$$\begin{aligned} \sum_{i=1}^n \sum_{k=1}^n \sum_{j=1}^n |a_{ij}||b_{jk}| &= \sum_{i=1}^n \sum_{j=1}^n |a_{ij}| \left(\sum_{k=1}^n |b_{jk}| \right) \\ &= \sum_{i=1}^n \sum_{j=1}^n |a_{ij}| \cdot \sum_{k=1}^n \sum_{j=1}^n |b_{jk}| \\ &= \left(\sum_{i=1}^n \sum_{j=1}^n |a_{ij}| \right) \left(\sum_{j=1}^n \sum_{k=1}^n |b_{jk}| \right) \\ &= \|A\| \cdot \|B\| \end{aligned}$$

Step 5: Combining Steps 3 and 4, we obtain: $\|AB\| \leq \|A\|\|B\|$.

Yes, it DOES conform to Inequality (10).

Problem 5

Problem Statement (Textbook 4.4.11):

Show that for the vector norm $\|\mathbf{x}\|_1$ defined in Equation (5), the subordinate matrix norm is

$$\|A\|_1 = \max_{1 \leq j \leq n} \sum_{i=1}^n |a_{ij}|$$

where

$$\|\mathbf{x}\|_1 = \sum_{i=1}^n |x_i| \quad (\text{Equation 5})$$

Solution:

We need to show that the subordinate matrix norm induced by $\|\mathbf{x}\|_1$ is the maximum absolute column sum.

Recall that the subordinate matrix norm is defined as:

$$\|A\|_1 = \max_{\mathbf{x} \neq \mathbf{0}} \frac{\|A\mathbf{x}\|_1}{\|\mathbf{x}\|_1}$$

Equivalently, we can write:

$$\|A\|_1 = \max_{\|\mathbf{x}\|_1=1} \|A\mathbf{x}\|_1$$

Part 1: Upper Bound ($\|A\|_1 \leq \max_j \sum_i |a_{ij}|$)

For any vector $\mathbf{x}$ with $\|\mathbf{x}\|_1 = 1$, we compute:

$$\begin{aligned}
 \|A\mathbf{x}\|_1 &= \sum_{i=1}^n \left| \sum_{j=1}^n a_{ij} x_j \right| \\
 &\leq \sum_{i=1}^n \sum_{j=1}^n |a_{ij}| |x_j| \quad (\text{triangle inequality}) \\
 &= \sum_{j=1}^n |x_j| \sum_{i=1}^n |a_{ij}| \quad (\text{interchange sums}) \\
 &\leq \sum_{j=1}^n |x_j| \cdot \max_{1 \leq k \leq n} \sum_{i=1}^n |a_{ik}| \\
 &= \left(\sum_{j=1}^n |x_j| \right) \cdot \max_{1 \leq k \leq n} \sum_{i=1}^n |a_{ik}| \\
 &= \|\mathbf{x}\|_1 \cdot \max_{1 \leq k \leq n} \sum_{i=1}^n |a_{ik}| \\
 &= \max_j \sum_{i=1}^n |a_{ij}|
 \end{aligned}$$

This holds for all $\mathbf{x}$ with $\|\mathbf{x}\|_1 = 1$, so:

$$\|A\|_1 \leq \max_j \sum_{i=1}^n |a_{ij}|$$

Part 2: Lower Bound (Equality is Achieved)

Let j^* be the column index that achieves the maximum:

$$j^* = \arg \max_{1 \leq j \leq n} \sum_{i=1}^n |a_{ij}|$$

Now choose the test vector $\mathbf{x}^* = \mathbf{e}_{j^*}$ (the j^* -th standard basis vector), which has $\|\mathbf{x}^*\|_1 = 1$.

Then:

$$A\mathbf{x}^* = A\mathbf{e}_{j^*} = \begin{bmatrix} a_{1,j^*} \\ a_{2,j^*} \\ \vdots \\ a_{n,j^*} \end{bmatrix}$$

Thus:

$$\|A\mathbf{x}^*\|_1 = \sum_{i=1}^n |a_{i,j^*}| = \max_j \sum_{i=1}^n |a_{ij}|$$

This shows that the upper bound is achieved, so:

$$\|A\|_1 \geq \max_j \sum_{i=1}^n |a_{ij}|$$

Conclusion

Combining Parts 1 and 2:

$$\|A\|_1 = \max_{1 \leq j \leq n} \sum_{i=1}^n |a_{ij}|$$

The subordinate matrix norm induced by the 1-norm is the **maximum absolute column sum**.

Problem 6

Problem Statement (Textbook 4.4.33):

For any $n \times n$ matrix A , define

$$\|A\|_F = \left(\sum_{i=1}^n \sum_{j=1}^n a_{ij}^2 \right)^{1/2}$$

(This is called the **Frobenius** norm.) Is this a subordinate matrix norm? Answer the same question for $\|A\| = \max_{1 \leq i, j \leq n} |a_{ij}|$. Prove that these equations define norms on the vector space of all $n \times n$ matrices.

Solution:

We prove both are valid norms, then check if they are subordinate.

Part 1: Frobenius Norm is a Valid Norm

We show $\|A\|_F = \sqrt{\sum_{i,j} a_{ij}^2}$ satisfies the three norm axioms.

Axiom 1: $\|A\|_F = 0$ iff $A = 0$

Since $\|A\|_F$ is a square root of squared terms, it's always ≥ 0 . If $\|A\|_F = 0$, then $\sum_{i,j} a_{ij}^2 = 0$. Since each term is squared (non-negative), all terms must be zero, so $a_{ij} = 0$ for all i, j .

Axiom 2: $\|\alpha A\|_F = |\alpha| \|A\|_F$

When we multiply by scalar α , every entry becomes αa_{ij} :

$$\|\alpha A\|_F = \sqrt{\sum_{i,j} (\alpha a_{ij})^2} = \sqrt{\alpha^2 \sum_{i,j} a_{ij}^2} = |\alpha| \sqrt{\sum_{i,j} a_{ij}^2} = |\alpha| \|A\|_F$$

Axiom 3: $\|A + B\|_F \leq \|A\|_F + \|B\|_F$

Think of the Frobenius norm as measuring the "length" of all entries. If we list all n^2 entries of A as a single long vector, then $\|A\|_F$ is just the Euclidean length of that vector. Since the Euclidean norm satisfies the triangle inequality, so does $\|\cdot\|_F$.

Therefore, the Frobenius norm is a valid norm.

Part 2: Max-Entry Norm is a Valid Norm

We show $\|A\|_{\max} = \max_{i,j} |a_{ij}|$ satisfies the three norm axioms.

Axiom 1: $\|A\|_{\max} = 0$ iff $A = 0$

The maximum absolute value is zero only if all entries have absolute value zero, meaning $a_{ij} = 0$ for all i, j .

Axiom 2: $\|\alpha A\|_{\max} = |\alpha| \|A\|_{\max}$

The largest entry in αA is $|\alpha|$ times the largest entry in A :

$$\|\alpha A\|_{\max} = \max_{i,j} |\alpha a_{ij}| = |\alpha| \max_{i,j} |a_{ij}| = |\alpha| \|A\|_{\max}$$

Axiom 3: $\|A + B\|_{\max} \leq \|A\|_{\max} + \|B\|_{\max}$

For each entry: $|a_{ij} + b_{ij}| \leq |a_{ij}| + |b_{ij}|$ (triangle inequality for numbers).

The largest entry in $A + B$ is bounded:

$$|a_{ij} + b_{ij}| \leq |a_{ij}| + |b_{ij}| \leq \max |a_{kl}| + \max |b_{mn}| = \|A\|_{\max} + \|B\|_{\max}$$

Therefore, the max-entry norm is a valid norm.

Part 3: Is Frobenius Norm Subordinate?

Answer: NO

A subordinate norm must satisfy $\|I\| = 1$ for the identity matrix (since $I\mathbf{x} = \mathbf{x}$, the matrix doesn't stretch any vector).

However, the identity matrix in Frobenius norm is:

$$\|I_n\|_F = \sqrt{1^2 + 1^2 + \dots + 1^2} = \sqrt{n}$$

(There are n ones on the diagonal.)

For $n \geq 2$, we have $\sqrt{n} > 1$, so the Frobenius norm is **not subordinate**.

Part 4: Is Max-Entry Norm Subordinate?

Answer: NO

Consider the $n \times n$ matrix with all entries equal to 1:

$$A = \begin{bmatrix} 1 & 1 & \dots & 1 \\ 1 & 1 & \dots & 1 \\ \vdots & \vdots & \ddots & \vdots \\ 1 & 1 & \dots & 1 \end{bmatrix}$$

The max-entry norm is:

$$\|A\|_{\max} = 1$$

But if we multiply by vector $\mathbf{x} = [1, 0, \dots, 0]^T$, we get:

$$A\mathbf{x} = [1, 1, \dots, 1]^T$$

If the max-entry norm were subordinate, we'd need:

$$\|A\|_{\max} \geq \frac{\|A\mathbf{x}\|}{\|\mathbf{x}\|}$$

But $A\mathbf{x}$ has n entries all equal to 1, so in any reasonable vector norm, $\|A\mathbf{x}\|$ grows with n . Meanwhile $\|\mathbf{x}\| = 1$, so the ratio $\frac{\|A\mathbf{x}\|}{\|\mathbf{x}\|}$ can be arbitrarily large as n increases. This contradicts $\|A\|_{\max} = 1$.

Therefore, the max-entry norm is **not subordinate**.

Summary

Property	Frobenius Norm	Max-Entry Norm
Valid norm?	Yes	Yes
Subordinate?	No ($\ I\ _F = \sqrt{n} \neq 1$)	No

Problem 7

Problem Statement (Textbook 4.6.1b):

Program the Gauss-Seidel method and test it on the following system:

$$\begin{aligned}3x + y + z &= 5 \\3x + y - 5z &= -1 \\x + 3y - z &= 3\end{aligned}$$

Analyze what happens when these systems are solved by simple Gaussian elimination without pivoting.

Solution:

System Matrix:

$$A = \begin{bmatrix} 3 & 1 & 1 \\ 3 & 1 & -5 \\ 1 & 3 & -1 \end{bmatrix}, \quad b = \begin{bmatrix} 5 \\ -1 \\ 3 \end{bmatrix}$$

True Solution (using direct method): $x = y = z = 1$

Part 1: Gauss-Seidel Method

The Gauss-Seidel iteration is:

$$\begin{aligned}x^{(k+1)} &= \frac{1}{3}(5 - y^{(k)} - z^{(k)}) \\y^{(k+1)} &= \frac{1}{1}(-1 - 3x^{(k+1)} + 5z^{(k)}) \\z^{(k+1)} &= \frac{1}{-1}(3 - x^{(k+1)} - 3y^{(k+1)})\end{aligned}$$

Diagonal Dominance Check:

- Row 1: $|a_{11}| = 3 > |1| + |1| = 2$
- Row 2: $|a_{22}| = 1 \not> |3| + |-5| = 8$
- Row 3: $|a_{33}| = |-1| = 1 \not> |1| + |3| = 4$

Matrix is NOT strictly diagonally dominant

Convergence Results (see hw05_p07.jl):

- Starting from $(1, 1, 1)$: **Converges in 1 iteration** (already at solution!)
- Starting from $(0.999, 0.999, 0.999)$: **Diverges** (NaN)
- Starting from $(1.01, 1.01, 1.01)$: **Diverges** (NaN)

- Starting from (0.9, 0.9, 0.9) or (1.1, 1.1, 1.1): **Diverges** (NaN)
- Starting from (0, 0, 0), (5, -1, 3), or (10, 10, 10): **Diverges** (NaN)

Key Observation: The method only "converges" when starting exactly at the solution. Even tiny perturbations (0.001 away) cause catastrophic divergence to NaN. This demonstrates **complete instability**.

Part 2: Gaussian Elimination without Pivoting

Forward elimination proceeds:

Step 1: Eliminate column 1 using pivot $a_{11} = 3$:

$$\begin{bmatrix} 3 & 1 & 1 \\ 0 & 0 & -6 \\ 0 & 8/3 & -4/3 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} 5 \\ -6 \\ 4/3 \end{bmatrix}$$

Step 2: Attempt to eliminate column 2 using pivot a_{22} :

$a_{22} = 0 \text{ (ZERO PIVOT!)}$

Analysis:

- Gaussian elimination **fails** due to zero pivot at position (2, 2)
- Rows 2 and 3 need to be swapped (partial pivoting required)
- Without pivoting, the method cannot proceed
- The system has determinant $\det(A) = 48 \neq 0$, so the matrix is invertible
- Failure is due to *method limitation*, not matrix singularity

Summary: Both methods encounter difficulties with this system:

- **Gauss-Seidel:** Diverges for most starting points (not diagonally dominant)
- **Gaussian Elimination:** Fails without pivoting (zero pivot encountered)

Implementation: See Appendix A for complete Julia code listings:

- Gauss-Seidel method: Appendix
- Gaussian elimination: Appendix

Problem 8

Problem Statement (Textbook 6.1.1d):

Find the polynomial of least degree that interpolates these sets of data:

$$\begin{array}{cccc} x & 3 & 7 & 12 \\ y & 12 & 146 & 2 \end{array}$$

Solution:

We find the polynomial of least degree interpolating the data points $(3, 12)$, $(7, 146)$, and $(12, 2)$.

Since we have 3 data points, the interpolating polynomial has degree at most $n - 1 = 2$ (quadratic).

Method: Vandermonde Matrix

We seek a polynomial $P(x) = a_0 + a_1x + a_2x^2$ such that $P(x_i) = y_i$ for $i = 1, 2, 3$.

This gives the linear system (see Textbook p. 313, Equation 12) [1]:

$$\begin{bmatrix} 1 & x_1 & x_1^2 \\ 1 & x_2 & x_2^2 \\ 1 & x_3 & x_3^2 \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \end{bmatrix} = \begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix}$$

Substituting our data:

$$\begin{bmatrix} 1 & 3 & 9 \\ 1 & 7 & 49 \\ 1 & 12 & 144 \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \end{bmatrix} = \begin{bmatrix} 12 \\ 146 \\ 2 \end{bmatrix}$$

Solving this system, we obtain:

$$\begin{aligned} a_0 &= -\frac{4211}{18} \approx -233.8667 \\ a_1 &= \frac{1849}{18} \approx 102.7222 \\ a_2 &= -\frac{125}{18} \approx -6.9222 \end{aligned}$$

Implementation: See Appendix B (??) for Julia code implementation.

Final Answer:

$$P(x) = -\frac{125}{18}x^2 + \frac{1849}{18}x - \frac{4211}{18}$$

Or in decimal form:

$$P(x) = -6.9222x^2 + 102.7222x - 233.8667$$

Verification:

- $P(3) = 12.000$

- $P(7) = 146.000$
- $P(12) = 2.000$

Maximum interpolation error: 5.68×10^{-14} (numerical precision).

Problem 9

Problem Statement (Textbook 6.1.8):

Discuss the problem of determining a polynomial of degree at most 2 for which $p(0)$, $p(1)$, and $p(\xi)$ are prescribed, ξ being any preassigned point.

Solution:

We seek a polynomial $p(x) = a_0 + a_1x + a_2x^2$ (degree at most 2) such that:

$$p(0) = y_0, \quad p(1) = y_1, \quad p(\xi) = y_\xi$$

where y_0, y_1, y_ξ are prescribed values and ξ is a given point.

Main Case: Three Distinct Points Required

For a polynomial of degree at most 2, we need three distinct interpolation points. Thus, we require:

$$\xi \neq 0 \text{ and } \xi \neq 1$$

The Vandermonde system is:

$$\begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 1 \\ 1 & \xi & \xi^2 \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \end{bmatrix} = \begin{bmatrix} y_0 \\ y_1 \\ y_\xi \end{bmatrix}$$

The determinant is:

$$\det(V) = \xi(\xi - 1)$$

Since $\xi \neq 0$ and $\xi \neq 1$, we have $\det(V) \neq 0$, so the system has a **unique solution**.

Conclusion: When $\xi \notin \{0, 1\}$, there exists a **unique polynomial of degree at most 2** satisfying all three conditions.

Degenerate Cases

Note: If $\xi = 0$ or $\xi = 1$, we only have two distinct interpolation points, which contradicts the requirement for a degree-2 polynomial. In these cases:

- The problem reduces to linear interpolation (degree at most 1)
- The prescribed values must be consistent (e.g., if $\xi = 0$, then $y_0 = y_\xi$)
- The resulting polynomial is the line through the two distinct points

These cases defeat the original question's intent of determining a degree-2 polynomial.

Key Insight: The problem is well-posed if and only if we have three distinct interpolation points, i.e., $\xi \notin \{0, 1\}$.

Appendix

A. Problem 7 Code Listings

A.1 Gauss-Seidel Method Implementation

```

73  function gauss_seidel(A, b, x0; max_iter=100, tol=1e-10)
74      n = length(b)
75      x = copy(x0)
76      history = [copy(x)]
77
78      for iter in 1:max_iter
79          x_old = copy(x)
80
81          # Gauss-Seidel iteration
82          for i in 1:n
83              sum_val = b[i]
84              for j in 1:n
85                  if j ≠ i
86                      sum_val -= A[i, j] * x[j]
87                  end
88              end
89              x[i] = sum_val / A[i, i]
90          end
91
92          push!(history, copy(x))
93
94          # Check convergence
95          error = norm(x - x_old, Inf)
96          if error < tol
97              return (x, true, iter, history)
98          end
99      end
100
101      return (x, false, max_iter, history)
102  end

```

Figure 1: Julia code implementing the Gauss-Seidel iterative method for Problem 7.

A.2 Gaussian Elimination without Pivoting

```

166 ✓ function gaussian_elimination_no_pivot(A, b)
167     n = length(b)
168     A_work = copy(A)
169     b_work = copy(b)
170
171     # Forward elimination
172 ✓   for k in 1:n-1
173       println(@sprintf("\nStep %d: Eliminate column %d", k, k))
174
175 ✓       if abs(A_work[k, k]) < 1e-14
176           println(" ⚠ WARNING: Pivot element is near zero!")
177           println(@sprintf("    a_%d%d = %.2e", k, k, A_work[k, k]))
178           return (zeros(n), false, A_work, b_work)
179       end
180
181       println(@sprintf("    Pivot: a_%d%d = %.6f", k, k, A_work[k, k]))
182
183 ✓       for i in k+1:n
184 ✓           if abs(A_work[i, k]) > 1e-14
185               multiplier = A_work[i, k] / A_work[k, k]
186               println(@sprintf("    Row %d: subtract %.6f × Row %d", i,
187                               multiplier, k))
187
188               A_work[i, k:n] -= multiplier * A_work[k, k:n]
189               b_work[i] -= multiplier * b_work[k]
190           end
191       end
192
193       println("\n Matrix after step $k:")
194       display(A_work)
195       println("\n RHS after step $k:")
196       display(b_work)
197   end

```

Figure 2: Julia code implementing Gaussian elimination without pivoting (Part 1).

```

199     # Back substitution
200     x = zeros(n)
201 ✓   for i in n:-1:1
202       x[i] = (b_work[i] - dot(A_work[i, i+1:n], x[i+1:n])) / A_work[i, i]
203   end
204
205   return (x, true, A_work, b_work)
206 end

```

Figure 3: Julia code implementing Gaussian elimination without pivoting (Part 2).

References

- [1] David Kincaid and Ward Cheney. *Numerical Analysis: Mathematics of Scientific Computing*. American Mathematical Society, Providence, RI, 3rd edition, 2009. Available from ProQuest Ebook Central. <https://ebookcentral.proquest.com/lib/wsu/detail.action?docID=4717637>.
- [2] Alexander Panchenko. Math 448 lecture notes. Washington State University, Fall 2025.